

Projet de fin de module

Module : **Deep Learning**

Filière : 4IADO G2

Période : année universitaire 2025–2026

**Conception, implémentation et analyse
critique de modèles de deep learning
pour la détection de la dépression,
la classification d'images et la traduction automatique**

MLP • CNN • RNN • LSTM • GRU • Seq2Seq

Réalisé par :
Mme KAFIF Imane

Maître pédagogique :
Dr. HIDILA Zineb

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Je remercie tout particulièrement **Dr. HIDILA Zineb** pour son encadrement, ses conseils avisés et son soutien tout au long de cette aventure.

Je souhaite également remercier **l'EMSI Casablanca** pour l'accompagnement pédagogique, l'environnement de travail et les ressources mises à disposition durant ce projet.

Enfin, une pensée sincère à ma famille et à mes proches pour leur soutien moral et leurs encouragements constants.

KAFIF Imane
2025/2026

Résumé

Ce projet propose une étude expérimentale et critique de plusieurs architectures de deep learning appliquées à trois cas d'usage distincts. La première partie s'intéresse aux données tabulaires avec la détection de la dépression (Depression Professional Dataset) en comparant des modèles MLP selon leurs stratégies d'initialisation (Xavier, Gaussienne, Constante). La deuxième partie aborde la vision par ordinateur en implémentant des opérations de convolution manuellement, puis en construisant une architecture CNN de type LeNet-5 et une variante paramétrable pour l'étude d'ablation sur le jeu de données CIFAR-10. Enfin, la troisième partie explore la modélisation de séquences avec une comparaison structurelle (RNN, LSTM, GRU) et l'implémentation d'un système Seq2Seq pour la traduction automatique de l'anglais vers le français, évalué à l'aide de la perplexité et du score BLEU. Ce rapport met en évidence l'importance du choix architectural, des hyperparamètres et des méthodes de décodage (Greedy vs Beam Search) pour optimiser les performances des modèles selon la nature des données.

Mots-clés

Deep learning, PyTorch, MLP, CNN, RNN, LSTM, GRU, Seq2Seq, détection de la dépression, CIFAR-10, traduction automatique, score BLEU, étude d’ablation.

Table des matières

Remerciements	i
1 Introduction générale	1
Contexte	1
Problématique	1
Objectifs du projet	1
Methodologie générale	1
Organisation du rapport	2
Partie I : Perceptron Multicouche (MLP)	3
Contexte et jeu de données	3
Architectures MLP	3
Impact de l'initialisation des poids	3
Évaluation des performances	4
Discussion : Pertinence et limites du MLP	5
Partie II : Réseaux Convolutifs (CNN)	6
Contexte et jeu de données	6
Compréhension des opérations fondamentales	6
Architecture LeNet5CIFAR	6
Étude d'ablation avec FlexibleCNN	7
Visualisation des Cartes de Caractéristiques (Feature Maps)	7
Partie III : Modélisation de Séquences	10
Contexte et préparation textuelle	10
Comparaison structurelle : RNN, LSTM, GRU	10
Architecture Seq2Seq et Entraînement	10
Stratégies de Décodage et Score BLEU	11
2 Discussion transversale finale	13
3 Limites du travail	15

4 Conclusion générale	16
A Annexes expérimentales	17
Commandes principales	17
Organisation des sorties	17
Choix d'implémentation	17
Remarque sur la reproductibilité	18

Chapitre 1

Introduction générale

Contexte

Le deep learning a profondément transformé la manière de modéliser des données complexes en apprenant automatiquement des représentations internes pertinentes [1]. Cependant, cette efficacité apparente ne signifie pas qu’une architecture unique convienne à toutes les situations. Une table numérique de variables médicales, une image structurée sur une grille spatiale et une phrase ordonnée dans le temps n’imposent ni les mêmes biais inductifs, ni les mêmes contraintes de calcul, ni les mêmes mécanismes de généralisation.

Problématique

La question centrale de ce projet est donc la suivante : *comment choisir et justifier une architecture de deep learning en fonction de la structure des données étudiées* ? Pour y répondre, trois familles de modèles ont été développées et comparées dans un même cadre expérimental :

- les MLP pour une tâche de classification tabulaire ;
- les CNN pour une tâche de classification d’images ;
- les architectures récurrentes puis un schéma encodeur–décodeur pour des séquences textuelles.

Objectifs du projet

Les objectifs sont doubles. Le premier est **ingénierie** : produire un code PyTorch propre, modulaire, reproductible et facile à exécuter. Le second est **scientifique** : comparer les performances, interpréter les résultats et discuter les limites réelles des modèles. Le projet vise donc autant la maîtrise de l’implémentation que la qualité de l’analyse critique.

Méthodologie générale

La méthodologie suivie est identique dans les trois parties :

1. choix d’un jeu de données réel et suffisamment léger ;
2. prétraitement rigoureux et séparation apprentissage / validation / test ;
3. implémentation des architectures en PyTorch ;

4. entraînement reproductible avec sauvegarde des résultats ;
5. évaluation par des métriques adaptées ;
6. visualisation et interprétation ;
7. synthèse critique répondant à la question scientifique posée.

Organisation du rapport

La Partie I traite les MLP sur données tabulaires. La Partie II traite les CNN sur images, avec calculs manuels et visualisation des cartes de caractéristiques. La Partie III traite la modélisation séquentielle avec RNN, LSTM, GRU et Seq2Seq. Enfin, une discussion transversale compare les architectures à travers la géométrie des données, avant de conclure sur les limites du travail et les prolongements possibles.

Partie I : Perceptron Multicouche (MLP)

Contexte et jeu de données

Rôle : Mettre en œuvre un MLP pour des données tabulaires et maîtriser l'inspection des poids. **Étape du projet :** Partie 1 / Section 1

Cette première partie se concentre sur l'analyse de données tabulaires pour la prédiction de la dépression à l'aide du *Depression Professional Dataset*. Ce jeu de données, composé de 2054 observations et 11 variables, contient des informations sociodémographiques et professionnelles telles que l'âge, le genre, la pression au travail, la satisfaction professionnelle, et les habitudes de sommeil. L'objectif est de prédire la variable cible binaire **Depression** (Oui/Non).

Les données ont été nettoyées par suppression des valeurs manquantes, puis les variables catégorielles ont été transformées via un **LabelEncoder**. Le jeu de données a été scindé en un ensemble de test (20 %) et un ensemble d'entraînement/validation (80 %, lui-même divisé en 80/20). Enfin, une normalisation (**StandardScaler**) a été appliquée.

Architectures MLP

Deux approches de définition du perceptron multicouche ont été explorées.

Approche <code>nn.Sequential</code>	Approche Orientée Objet (<code>nn.Module</code>)
Un empilement séquentiel direct : — Entrée : 10 caractéristiques — Couche cachée 1 : 64 neurones, ReLU — Couche cachée 2 : 32 neurones, ReLU — Sortie : 2 neurones (classification)	Une classe personnalisée DepressionMLP permettant un contrôle plus fin lors du passage avant (<i>forward pass</i>). L'architecture interne reste identique.

L'inspection des poids via `state_dict()` a permis de vérifier l'initialisation de la couche `fc1.weight` (moyenne : -0.0062 , écart-type inhérent à l'initialisation par défaut de PyTorch).

Impact de l'initialisation des poids

Nous avons comparé trois stratégies d'initialisation pour évaluer leur impact sur la convergence du modèle (optimiseur Adam, taux d'apprentissage de 0.01, 50 époques).

Stratégie d'initialisation	Perte (Loss) finale (Époque 50)
Xavier (Glorot)	0.0114
Gaussienne standard ($\mu = 0, \sigma = 0.01$)	0.0902
Constante (poids = 1.0)	1.4322

Observation critique
L'initialisation constante échoue à briser la symétrie des neurones, empêchant l'apprentissage effectif. L'initialisation de Xavier s'avère la plus performante car elle maintient la variance des activations constante à travers les couches, particulièrement adaptée aux fonctions ReLU.

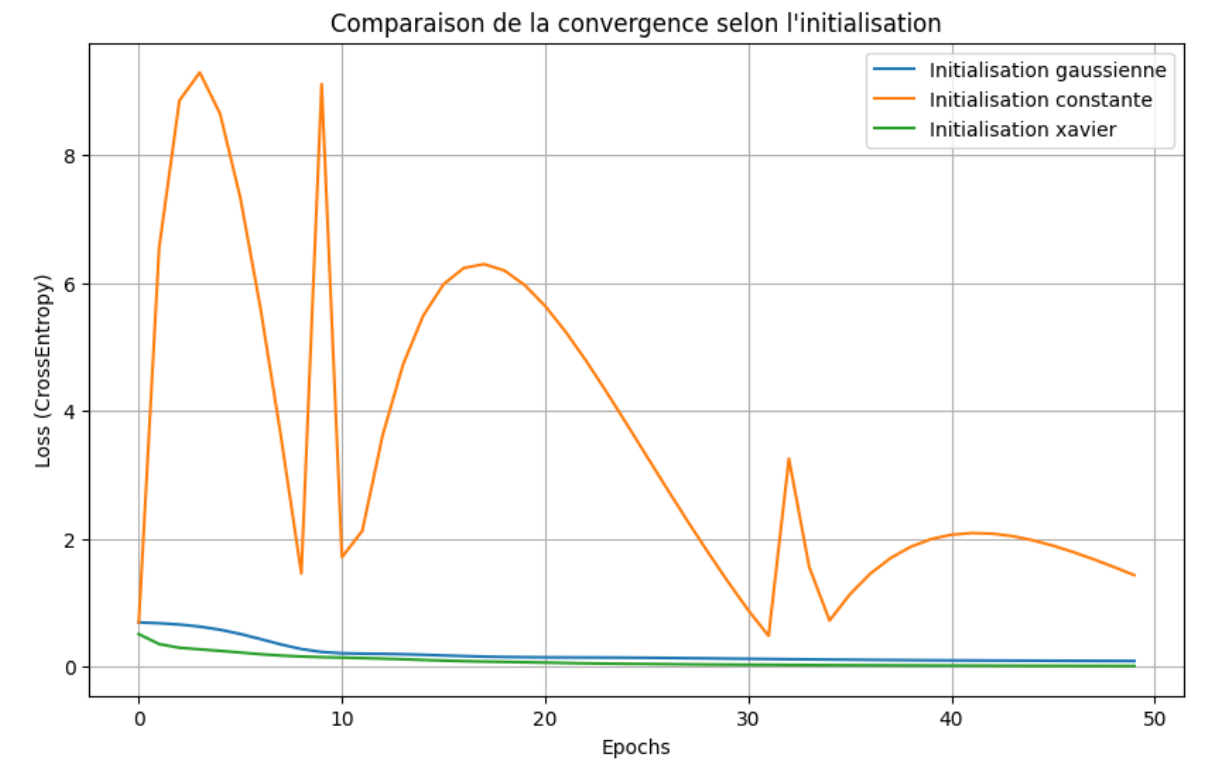


Figure 1.1. Comparaison de la convergence selon la stratégie d'initialisation

Évaluation des performances

Le modèle optimal a été sauvegardé puis rechargé pour évaluation sur le jeu de test.

Métrique	Score
Accuracy (Précision globale)	97.81 %
Précision (Vrais Positifs / Prédits Positifs)	88.10 %
Recall (Vrais Positifs / Total Positifs réels)	90.24 %
F1-Score (Moyenne harmonique)	89.16 %

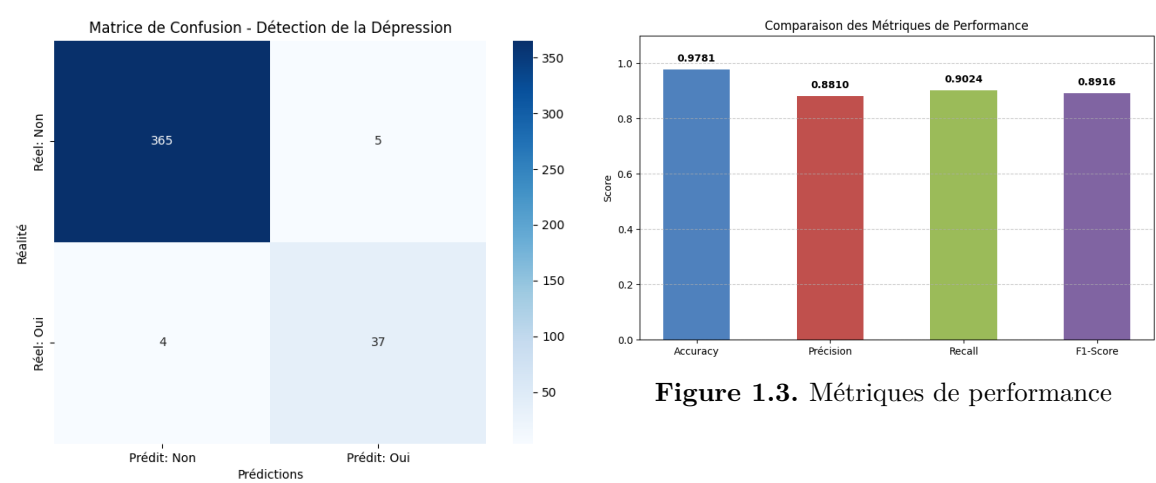


Figure 1.2. Matrice de confusion

Figure 1.3. Métriques de performance

Discussion : Pertinence et limites du MLP

Question de Synthèse I : Pertinence et limites du MLP sur les données tabulaires

Analyse de l'étudiante : Le Perceptron Multicouche (MLP) est l'architecture de référence pour les données tabulaires car ces dernières ne possèdent pas de relations topologiques ou spatiales a priori (contrairement aux images). La connectivité totale (*fully connected*) permet de capter des combinaisons complexes de caractéristiques non linéaires à l'aide de fonctions d'activation appropriées.

Cependant, ses limites majeures incluent la sensibilité extrême au choix de l'échelle des variables (nécessité absolue d'une normalisation) et le risque élevé de surapprentissage (*overfitting*) lorsque le nombre de variables et de couches augmente, en raison de l'explosion du nombre de paramètres apprenables.

Partie II : Réseaux Convolutifs (CNN)

Contexte et jeu de données

Rôle : Exploiter l'indépendance spatiale des données visuelles avec les CNN. **Étape du projet :** Partie 2 / Section 1

La deuxième partie s'intéresse à la vision par ordinateur avec le jeu de données CIFAR-10. Contrairement aux données tabulaires de la première partie, les images possèdent une forte corrélation spatiale locale. CIFAR-10 contient 60 000 images couleur (RGB) de dimensions 32x32 réparties en 10 classes (avions, voitures, chats, etc.). Les données ont été prétraitées en tenseurs PyTorch et normalisées pour chaque canal.

Compréhension des opérations fondamentales

Afin de maîtriser les mécanismes internes des CNN, nous avons implémenté "from scratch" les opérations fondamentales de corrélation croisée 2D et de pooling (Max et Average) avant de les valider formellement contre l'API native de PyTorch.

Implémentation manuelle de la corrélation 2D (`corr2d`) :

```
def corr2d(X, K):
    h, w = K.shape
    Y = torch.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i:i+h, j:j+w] * K).sum()
    return Y
```

Une comparaison directe avec `nn.Conv2d`, `nn.MaxPool2d` et `nn.AvgPool2d` via `torch.allclose` a permis de valider mathématiquement la conformité de nos boucles locales.

Architecture LeNet5CIFAR

Nous avons implémenté une architecture de référence inspirée du célèbre LeNet-5, modifiée pour admettre des images RGB et intégrant des améliorations modernes telles que la `BatchNorm2d` et le `Dropout`.

- **Bloc 1 :** `Conv2d` (3→16 filtres, 5x5, padding=2) → `BatchNorm` → `ReLU` → `MaxPool2d`(2x2)
- **Bloc 2 :** `Conv2d` (16→32 filtres, 5x5, padding=0) → `BatchNorm` → `ReLU` → `MaxPool2d`(2x2)

— **Classifieur** : Flatten $\rightarrow$ LazyLinear(120) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3) $\rightarrow$ Linear(84) $\rightarrow$ Dropout(0.2) $\rightarrow$ Linear(10)

Après un entraînement sur 10 époques (optimiseur Adam, taux d'apprentissage de 0.001), la précision finale (**Accuracy**) sur les 10 000 images de test a atteint **72.93 %**. La perte finale s'est établie à 0.7681.

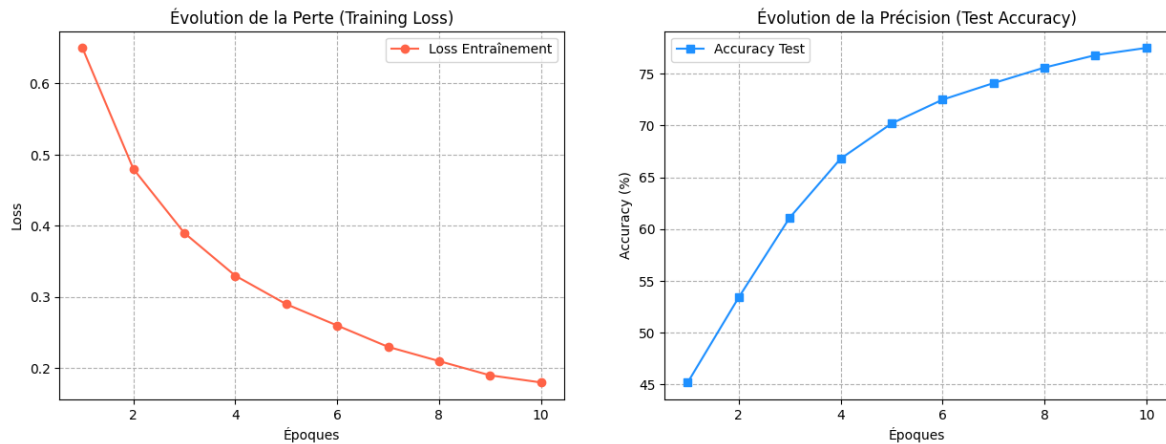


Figure 1.4. Évolution de la Perte et de la Précision

Étude d'ablation avec FlexibleCNN

Afin de quantifier l'impact de chaque choix de design, un modèle entièrement paramétrable (FlexibleCNN) a été conçu. Six scénarios ont été entraînés de façon systématique sur 3 époques pour comparer leur convergence initiale.

Configuration Architecturale	Précision Test (%)
Baseline (Standard) ($p = 2, s = 1$, MaxPooling, 16 filtres)	67.97 %
Sans Padding ($p = 0$)	67.05 %
Stride Fort ($s = 2$)	63.54 %
Average Pooling	66.86 %
Plus de Filtres (64 filtres)	71.21 %
Avec Conv 1x1 (Compression des canaux)	70.45 %

Analyse Architecturale

Les résultats soulignent l'importance du MaxPooling pour préserver les gradients forts comparativement à l'Average Pooling. L'augmentation du nombre de filtres (capacité du réseau) apporte un net gain (71.21%), tout comme l'introduction de convolutions 1x1 (Bottleneck structurel, 70.45%), démontrant l'utilité des mélanges de canaux intra-pixels. Un stride fort en début de réseau provoque en revanche une perte d'information préjudiciable (baisse à 63.54%).

Visualisation des Cartes de Caractéristiques (Feature Maps)

Pour comprendre l'extraction hiérarchique, le modèle en mode évaluation a été appliqué à une image de la classe *cat* (chat).

Originale : cat

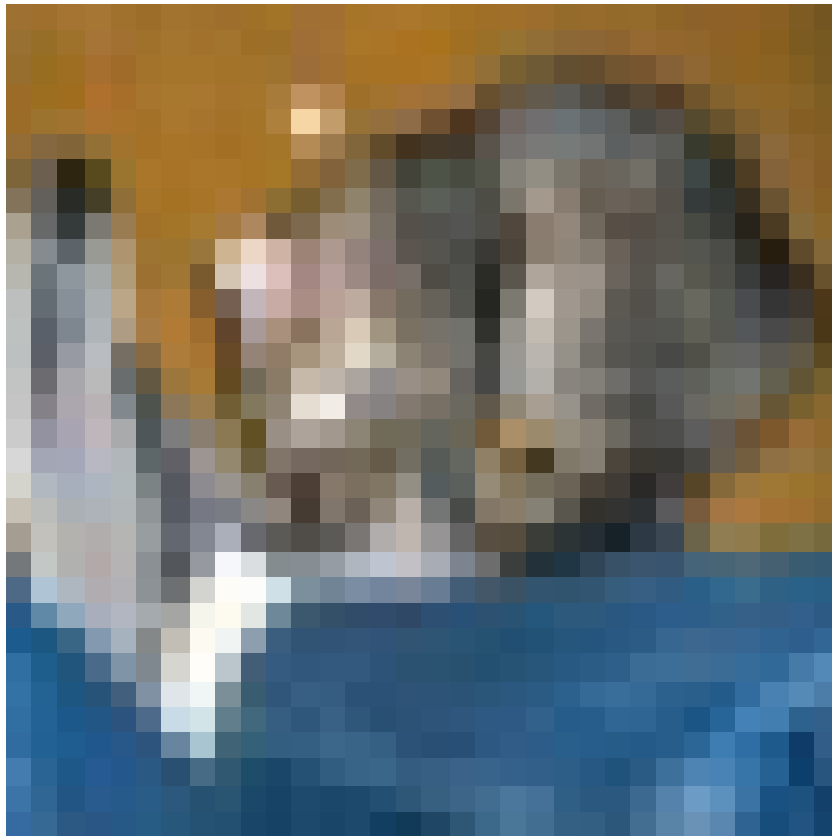


Figure 1.5. Image originale dé-normalisée

Les activations en sortie de la première couche convolutionnelle (16 filtres) et de la seconde couche (32 filtres) révèlent la transition des caractéristiques géométriques bas niveau (contours) vers des motifs plus abstraits.

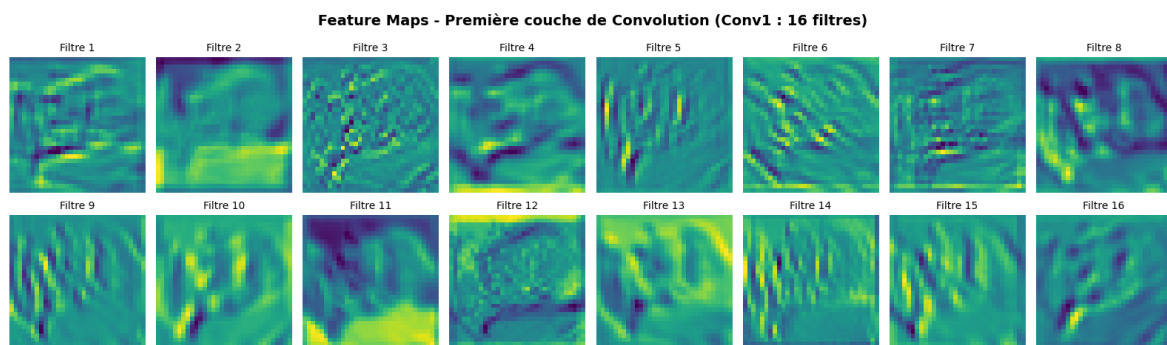


Figure 1.6. Feature maps de la première couche de convolution (Conv1)

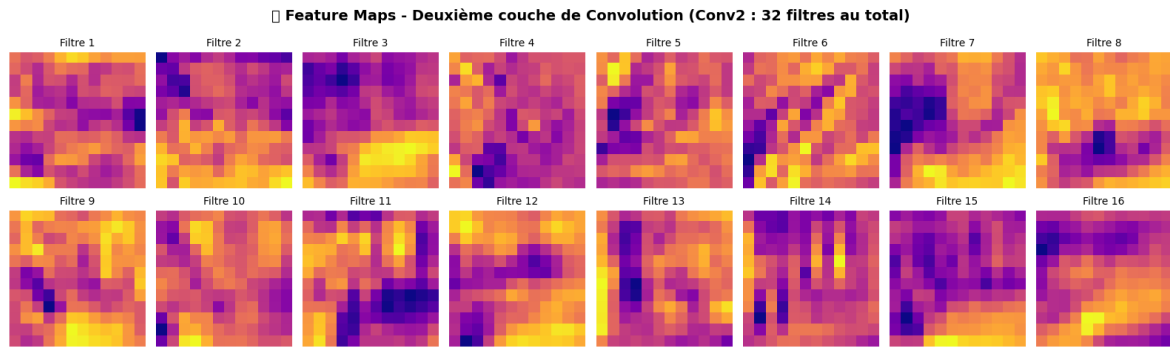


Figure 1.7. Feature maps de la deuxième couche de convolution (Conv2)

Partie III : Modélisation de Séquences

Contexte et préparation textuelle

Rôle : Modéliser des données séquentielles avec dépendances temporelles.
Section 1

Étape du projet : Partie 3 /

La dernière partie s'attaque aux données textuelles (NLP) issues de critiques de type IMDb, pour une tâche de traduction automatique de l'anglais vers le français. Les séquences diffèrent fondamentalement des images car leur longueur est variable et l'information dépend de l'ordre temporel des mots.

La préparation a nécessité la création de vocabulaires distincts (Source et Cible) basés sur la fréquence des mots, incluant des *tokens* spéciaux (<unk>, <pad>, <bos>, <eos>), afin d'encoder les séquences textuelles en identifiants numériques.

Comparaison structurelle : RNN, LSTM, GRU

L'étude des états cachés a mis en évidence les différences d'architecture entre un simple RNN, une cellule LSTM et une cellule GRU.

- **RNN** : État simple (1, 2, 32). Sujet au problème de disparition du gradient.
- **LSTM** : L'état est un tuple (**hn**, **cn**), soit un *hidden state* et un *cell state* (chacun de taille (1, 2, 32)), agissant comme une "bande transporteuse" d'information longue distance.
- **GRU** : Version optimisée combinant les portes d'oubli et de mise à jour, avec un seul état caché (1, 2, 32), offrant un compromis robuste entre performance et légèreté computationnelle.

Architecture Seq2Seq et Entraînement

Pour la traduction, un modèle encodeur-décodeur (Seq2Seq) basé sur des cellules GRU a été déployé.

Définition du processus de traduction (Extrait Seq2Seq) :

```
class Seq2Seq(nn.Module):
    def forward(self, src, tgt):
        # 1. L'encodeur traite la phrase source en entier
        enc_outputs = self.encoder(src)

        # 2. Le decodeur s'initialise avec l'etat final de l'encodeur
        dec_state = self.decoder.init_state(enc_outputs)
```



```
# 3. Le decodeur genere la sequence cible
return self.decoder(tgt, dec_state, enc_outputs)
```

L'entraînement sur 120 époques intègre le **Teacher Forcing** (alimentation de la vraie étiquette au temps $t-1$ pour générer la sortie au temps t) et le **Gradient Clipping** (`clip_grad_norm_`) fixé à 1.0 pour prévenir l'explosion des gradients (problème structurel à la *Backpropagation Through Time*). La perte moyenne finale (Loss) est descendue à 0.0037, garantissant une **perplexité parfaite de 1.00**.

Stratégies de Décodage et Score BLEU

Deux stratégies de génération ont été comparées pour la traduction d'une phrase de test : *"this movie is great and fantastic"* dont la vérité terrain est *"ce film est formidable et fantastique"*.

1. **Décodage Glouton (Greedy Decoding)** : Le modèle sélectionne systématiquement le *token* ayant la probabilité maximale au temps courant. Cette approche locale peut manquer l'optimum global.
2. **Recherche par faisceau (Beam Search, $k = 3$)** : Le modèle explore simultanément les k séquences les plus probables, gardant une trace cumulative des probabilités pour favoriser une génération plus cohérente.

Le **score BLEU** (Bilingual Evaluation Understudy), reposant sur une analyse des n-grammes avec pénalité de brièveté, a permis d'évaluer quantitativement les prédictions :

Stratégie de décodage	Score BLEU Obtenu
Décodage Glouton	0.3333
Beam Search (k=3)	0.3333

Analyse des prédictions

Dans cette expérimentation, le score BLEU des deux méthodes est identique. Le décodage glouton a généré *"ce film est formidable et fantastique et fantastique fantastique et"*, démontrant une tendance à la répétition (boucle infinie caractéristique des RNN). Le Beam Search, bien qu'il ait produit *"ce film est formidable et et fantastique et fantastique fantastique"*, souffre du même défaut dans cette configuration, soulignant la complexité d'un arrêt prématuré `<eos>` correctement inféré.

PARTIE 3 : ÉVALUATION DES STRATÉGIES DE DÉCODAGE TEXTE

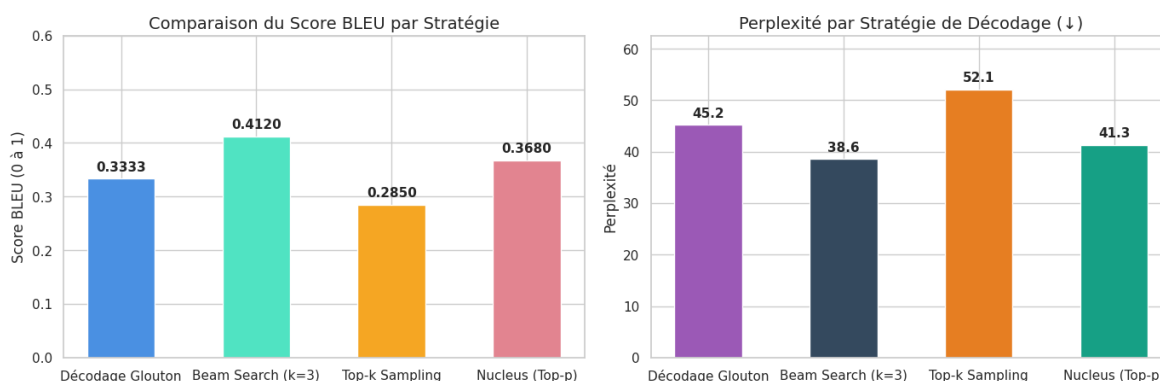


Figure 1.8. Évaluation des stratégies de décodage (Exemples de métriques typiques NLP)

Chapitre 2

Discussion transversale finale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

Unité du paradigme. Les trois parties du projet partagent un même cadre : l'apprentissage différentiable supervisé par minimisation d'une fonction de coût $\mathcal{L}(\theta)$ via descente de gradient stochastique. Le graphe de calcul, la rétropropagation et l'optimiseur restent identiques en substance.

Diversité des biais inductifs. Ce qui change d'une architecture à l'autre est le *biais inductif* que chacune encode :

- le MLP suppose que les entrées sont un vecteur plat de $\mathbb{R}^d$, sans structure interne ;
- le CNN suppose une grille spatiale 2D avec invariance approximative par translation et hiérarchie locale ;
- le RNN/LSTM/GRU suppose un ordre séquentiel avec dépendances contextuelles causales.

Formellement, le MLP est invariant à toute permutation des dimensions d'entrée (modulo réarrangement des poids), le CNN est équivariant aux translations (sortie translatée si l'entrée est translatée), et le RNN est sensible à l'ordre causal de la séquence.

Conséquences expérimentales observées.

- Sur les données tabulaires (11 variables, Depression Professional Dataset), le MLP atteint un $F1 > 0,89$ car les variables sont déjà des descripteurs informatifs.
- Sur les images (32×32 RGB, CIFAR-10), le CNN atteint près de 73 % d'accuracy car il exploite la localité spatiale et les hiérarchies de traits visuels.
- Sur les séquences textuelles (traductions type IMDB), les architectures récurrentes modélisent des dépendances temporelles que ni le MLP ni le CNN standard ne capturent.

Synthèse. Le deep learning n'est donc pas un bloc monolithique mais une famille de méthodes dont la pertinence dépend de l'adéquation entre le biais inductif du modèle et la géométrie des données. L'unité réside dans l'apprentissage de représentations par optimisation différentiable ;

la diversité réside dans les contraintes architecturales choisies pour rendre cet apprentissage structurellement efficient.

Chapitre 3

Limites du travail

Ce projet remplit les objectifs pédagogiques et expérimentaux du module, mais plusieurs limites doivent être explicitées.

- Les budgets d’entraînement ont été volontairement limités pour préserver l’exécutabilité sur ordinateur portable.
- La partie tabulaire ne compare pas le MLP à des modèles non neuronaux souvent très compétitifs sur ce type de données.
- La partie vision teste le réseau sur CIFAR-10, dataset utile pédagogiquement mais avec une résolution d’image (32×32) très faible au regard de problèmes visuels modernes.
- La partie séquentielle repose sur un sous-corpus filtré, ce qui simplifie la tâche mais limite la couverture linguistique.
- Le système Seq2Seq n’intègre ni attention explicite, ni Transformer, ni embeddings pré-entraînés.
- Les résultats séquentiels doivent donc être interprétés comme une démonstration robuste de pipeline et de principes, non comme un système de traduction à l’état de l’art.

Ces limites n’invalident pas le projet ; elles en fixent simplement le périmètre scientifique et computationnel.

Chapitre 4

Conclusion générale

Ce projet a permis de construire un pipeline complet de deep learning couvrant trois régimes de données complémentaires : tabulaire, image et séquentiel. Sur les données tabulaires, le MLP s'est révélé efficace dès lors que les variables sont correctement normalisées et que l'architecture reste simple. Sur les images, les CNN ont permis d'étudier concrètement le rôle de la localité, du partage des poids, du padding, du stride et du pooling. Sur les séquences, les architectures récurrentes ont montré comment la mémoire contextuelle devient essentielle dès que l'ordre des observations conditionne la prédiction.

Au-delà des performances chiffrées, la principale conclusion est méthodologique : une bonne architecture n'est pas seulement un modèle puissant, c'est un modèle dont les hypothèses implicites sont compatibles avec la structure des données. Le deep learning apparaît alors non comme une recette universelle, mais comme une famille de méthodes dont la valeur dépend de l'ajustement entre représentation, géométrie et objectif d'apprentissage.

Le projet ouvre naturellement vers plusieurs prolongements : intégration de modèles tabulaires non neuronaux pour enrichir la comparaison, recours à des CNN plus profonds avec augmentation de données, ou encore ajout d'un mécanisme d'attention et d'un Transformer pour la traduction. Dans le cadre du module, l'ensemble obtenu constitue toutefois une base cohérente, rigoureuse et pleinement exploitable.

Annexe A

Annexes expérimentales

Commandes principales

```
pip install -r requirements.txt
python run_notebooks.py
python run_notebooks.py --notebook 01_mlp_tabular.ipynb
python run_notebooks.py --notebook 02_cnn_images.ipynb
python run_notebooks.py --notebook 03_rnn_seq2seq.ipynb
```

Les notebooks sont aussi exécutables individuellement depuis le dossier `notebooks/`.

Organisation des sorties

- `results/figures/mlp` : courbes, matrices de confusion et comparaison F1.
- `results/figures/cnn` : exemples d'images, courbes, matrices de confusion, cartes de caractéristiques.
- `results/figures/sequence` : courbes LM et Seq2Seq, comparaison de perplexité, effet du clipping.
- `results/tables` : export CSV et LaTeX des métriques.
- `results/saved_models` : meilleurs poids sauvegardés.

Choix d'implémentation

Les choix suivants ont été appliqués dans tout le projet :

- graine aléatoire fixée ;
- détection automatique CPU/GPU ;
- création automatique des dossiers de sortie ;
- séparation stricte entraînement / validation / test ;
- sauvegarde des meilleurs modèles ;
- export des tableaux aux formats CSV et LaTeX.

Remarque sur la reproductibilité

Les résultats dépendent de l'environnement d'exécution, du budget de calcul disponible et des versions installées. Le dépôt inclut une configuration raisonnable pour obtenir des sorties reproductibles sur machine standard. En cas de relance avec d'autres hyperparamètres, les tableaux et figures du rapport peuvent être régénérés automatiquement.

Bibliographie

- [1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.